

Trabajo Práctico Individual - Ingeniería de Software II

Código: 20251C

Entrega

La resolución del presente proyecto es **individual**. Leer la totalidad del enunciado antes de comenzar con el desarrollo de la solución.

La entrega del proyecto se realizará mediante un repositorio privado en GitHub. Cada candidato deberá crear su propio repositorio y agregar a su corrector como colaborador para que pueda ser evaluado.

Recomendaciones:

- Iniciar tempranamente y hacer commits pequeños que agreguen funcionalidad incrementalmente
- No se evaluará ningún commit realizado luego de la fecha y hora límite de entrega

Contenido del Repositorio

El repositorio debe incluir un archivo **README.md** (en español) con:

- Una tabla de contenido
- Una introducción con no más de un párrafo pequeño y conciso sobre la solución planteada
- Una sección sobre qué fue lo más desafiante del proyecto
- Un apartado de pre-requisitos listando lo necesario para levantar el entorno de desarrollo, especificando los lenguajes y versiones de los manejadores de paquetes necesarios
- Link al "user-guide" de la librería que se usó para testear, o en su defecto link al repo (ej: Junit, gin-gonic)
- Comandos para construir la imagen de Docker
- Comandos para correr la base de datos
- Comandos para correr la imagen del servicio

Fechas Importantes

Fecha máxima de entrega: 20/03/2025

Formulario de Entrega

Se deberá completar el siguiente formulario con Padrón, Nombre, Apellido y Link al repositorio con la solución.

Formulario: [Link del formulario]

Luego de haber completado este formulario, les solicitaremos que agreguen al usuario del corrector asignado.

Criterios de Evaluación

Criterio	Descripción
Calidad del Código	Código limpio, legible y mantenible
Implementación	Implementación correcta de endpoints y features, siguiendo la especificación
Testing	Calidad de las pruebas para cada endpoint
Dockerfile	Creación adecuada del Dockerfile con las mejores prácticas
Configuración del Entorno	Uso de variables de entorno para configuraciones, sin codificar ningún valor
Manejo de Errores	Manejo correcto y estandarizado de errores usando RFC 7807
Desafíos (si los hay)	Implementación exitosa de cualquier desafío opcional
Documentación	README claro y conciso con todas las secciones requeridas
Proceso de Pensamiento	Explicación de decisiones y proceso de pensamiento a lo largo de la implementación

Enunciado

Muchas gracias por tu interés en la posición de **Founding Engineer en ClassConnect**. Estamos construyendo la próxima generación de plataformas de aprendizaje, y necesitamos mentes brillantes como la tuya para hacer realidad esta visión.

Tendrás **diez días** para completar esta prueba técnica. Si bien estimamos que el ejercicio debería tomarte entre **4-6 horas**, entendemos que puedas tener otros compromisos, así que queremos darte flexibilidad para que puedas demostrar tu mejor trabajo.

Si tienes alguna duda sobre los requerimientos o el alcance del proyecto, no dudes en contactarnos. Estamos aquí para ayudarte a brillar.

¡Esperamos ver tu solución! Buena suerte.

Historias de Usuario

1. Publicación de Cursos

Descripción: Como usuario de ClassConnect, quiero poder crear y publicar cursos para que otros usuarios puedan aprender.

Criterio de Aceptación: El sistema debe permitir a los usuarios crear y publicar cursos que incluyan un nombre y una descripción.

2. Visualizar Cursos

Descripción: Como usuario, quiero tener acceso a todos los cursos publicados en la plataforma.

Criterio de Aceptación: Cuando ingrese a la lista de cursos, se debe mostrar todos los cursos publicados en orden cronológico inverso.

Requisitos

Especificaciones del Servicio Backend

- El servicio debe ser una **API RESTful** y devolver datos en formato **JSON**
- Asegúrate de probar cada endpoint con al menos una prueba **E2E (End to End)** o de integración, de manera que las pruebas sean claras y descriptivas en su propósito

Endpoints a Implementar

Utiliza la siguiente especificación de **OpenAPI** para implementar los endpoints, que se puede visualizar en Swagger Editor para una vista más gráfica.

openapi: 3.0.0

info:

title: ClassConnect Content Service API

version: 1.0.0

paths:

/courses:

post:

summary: Create a new course

requestBody:

required: true

content:

application/json:

schema:

\$ref: '#/components/schemas/CreateCourseRequest'

responses:

'201':

description: Course created successfully

```

    content:
      application/json:
        schema:
          type: object
          properties:
            data:
              $ref: '#/components/schemas/Course'
  '400':
    description: Bad request error
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
get:
  summary: Retrieve all courses
  responses:
    '200':
      description: A list of courses
      content:
        application/json:
          schema:
            type: object
            properties:
              data:
                type: array
                items:
                  $ref: '#/components/schemas/Course'
/courses/{id}:
get:
  summary: Retrieve a course by ID
  parameters:
    - in: path
      name: id
      required: true
      schema:
        type: integer
  responses:
    '200':
      description: Course retrieved successfully
      content:
        application/json:
          schema:
            type: object
            properties:
              data:
                $ref: '#/components/schemas/Course'
    '404':
      description: Course not found

```

```
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
  '500':
    description: Internal server error
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
delete:
  summary: Delete a course by ID
  parameters:
    - in: path
      name: id
      required: true
      schema:
        type: integer
  responses:
    '204':
      description: Course deleted successfully
    '404':
      description: Course not found
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorResponse'
    '500':
      description: Internal server error
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorResponse'
components:
  schemas:
    Course:
      type: object
      properties:
        id:
          type: integer
        title:
          type: string
        description:
          type: string
    CreateCourseRequest:
      type: object
      properties:
        title:
```

```
    type: string
  description:
    type: string
ErrorResponse:
  type: object
  properties:
    type:
      type: string
    title:
      type: string
    status:
      type: integer
    detail:
      type: string
    instance:
      type: string
```

Importante: Las respuestas de error deben seguir el **RFC 7807**. Para este proyecto, por la complejidad, el campo `type` debe ser `about:blank`.

Persistencia de Datos

Los datos deben persistir durante la ejecución del programa; mantenerlo en memoria es suficiente dentro del scope solicitado.

Si se cuenta con el conocimiento para implementarlo, se recomienda persistir los datos en una **base de datos** para un manejo más estructurado.

Requisitos de CI/CD y DevOps

Uso de Variables de Entorno

Entorno de desarrollo: Utilizar variables de entorno para configurar parámetros básicos del servicio, como `HOST`, `PORT`, y `ENVIRONMENT`.

Persistencia: Utilizar las variables relacionadas con la conexión a bases de datos (`DATABASE_HOST`, `DATABASE_NAME`, `DATABASE_PORT`, `DATABASE_USER`, `DATABASE_PASSWORD`) si se emplea una base de datos. Si no aplica, omitir estas variables.

Aclaraciones:

- **ENVIRONMENT:** Define si el entorno es de desarrollo (`development`) o producción (`production`)
- **PORT:** Define el puerto en el que corre la aplicación, por defecto `8080`

- **HOST:** Define la IP donde el servicio escucha. `0.0.0.0` permite acceso desde cualquier red, por ejemplo, desde otro contenedor; `127.0.0.1` restringe a conexiones locales. La necesidad de definir HOST dependerá de la tecnología

Dockerfile

Crear un **Dockerfile** para el servicio siguiendo las mejores prácticas para una imagen concisa y eficiente.

Requerimientos No Funcionales

- Usa cualquier **lenguaje de programación** para el servicio, preferiblemente las últimas versiones LTS
- El servicio debe ejecutarse en un **contenedor de Docker**
- El **Dockerfile** debe estar en la raíz del repositorio
- Documenta funciones y clases siguiendo el estándar del lenguaje elegido
- Implementación de **logs**, utiliza una biblioteca externa de logging para facilitar la configuración
- Formatea el código con el formatter preferido del lenguaje
- El código debe estar escrito en **inglés**
- Los commits deben ser **atómicos y descriptivos**, para asegurar consistencia y legibilidad desde el inicio del proyecto

Desafíos Opcionales

1. Longitud Mínima y Máxima de la Descripción

Requiere que el campo `description` tenga al menos **50 caracteres** y como máximo **255 caracteres** al crear un curso. Responde con código `400` si la validación falla. Agrega un test para este caso.

2. UUID para Cursos

Asegura que cada curso tenga un **UUID v4**. El servidor debe generar automáticamente el UUID al momento de la creación de un curso, como respuesta de `POST /courses`.

El UUID debe manejarse internamente con la API o biblioteca nativa del lenguaje como un identificador de 128 bits. En el contrato REST, el UUID V4 debe representarse como un string en formato estándar y debe usarse como identificador en todas las operaciones en lugar del ID numérico.

3. Usar Middleware para Manejar Errores

Implementa middleware para el manejo centralizado de errores.

4. Mejoras a la Solución

¿Hay espacio para mejorar tu solución? Por favor, elabora.

5. Uso de Docker Compose

Agrega un archivo `compose.yaml` para definir los servicios. La base de datos y el contenedor de la aplicación deben estar definidos en `compose.yaml`. El servicio de Docker debe apuntar al Dockerfile para la construcción del backend.

Ejemplo de Respuesta de Error en Formato RFC 7807

Ejemplo de una respuesta de error usando el formato **RFC 7807**:

```
{
  "type": "https://example.com/errors/course-not-found",
  "title": "Course Not Found",
  "status": 404,
  "detail": "The course with ID 12345 was not found.",
  "instance": "/courses/12345"
}
```

Ejemplo de Lanzamiento de Contenedores para Base de Datos

PostgreSQL

```
docker run --name db-container --network <courses-network-name> -p 5432:5432 \
-e POSTGRES_DB=coursesdb \
-e POSTGRES_USER=courses.admin \
-e POSTGRES_PASSWORD=courses.password \
postgres:16.3-bookworm
```

MySQL

```
docker run --name db-container --network <courses-network-name> -p 3306:3306 \
-e MYSQL_DATABASE=coursesdb \
-e MYSQL_USER=courses.admin \
-e MYSQL_PASSWORD=courses.password \
-e MYSQL_ROOT_PASSWORD=root.password \
mysql:8.0.37-bookworm
```

Aclaraciones

Para asegurar la comunicación entre contenedores lanzados manualmente, primero crea una red de Docker con `docker network create <courses-network-name>` y luego

usa `--network <courses-network-name>` al lanzar los contenedores. De lo contrario, no podrán comunicarse.

Alternativamente, puedes usar **Docker Compose**, que los coloca automáticamente en la misma red.

Referencias

- **OpenAPI Specification**
- **RFC 7807**: Problem Details for HTTP APIs
- **Docker Documentation**
- **Conventional Commits**
- **The Twelve-Factor App**
- **UUID RFC**